AIML COURSE PROJECT · B.TECH 3RD YEAR

Vani-Setu

A real-time sign language translator using hand landmarks and machine learning

Project Report: Milestone 1 and Milestone 2

Team member	Roll number	Responsibility (vertical slice)
Abdullah Naseem	S24CSEU1193	Live Spell path: camera UI, classify API, SVM features
Bhanu Satwik	S24CSEU0898	Product path: login, translation, Phrase / ISL rules
Manoj Kumar K	S24CSEU1239	Data path: recording, training, practice mode, evaluation

Project guide	Dr. Mohd Anas
Department / Institute	School of Computer Science Engineering and Technology (CSET), Bennett University
Evaluation	Milestone 2, Week 8 (5–9 October 2026)
Live demo	vani-setu.onrender.com
Repository	github.com/ShadowAN-AB/Vani_Setu

Mapping to the evaluation rubric

Evaluation component	Marks	Where in this report
Problem domain identification and analysis (M1)	3	§1 Problem domain, analysis and problem statement
Review of existing systems and project feasibility	3	§2 Six existing systems, gap analysis, feasibility study
Objectives and methodology of the proposed work	3	§3 Seven objectives, seven-stage pipeline, dataset, evaluation plan
Relevance of algorithms / techniques	3	§4 Each algorithm, why it fits, results and experiments
Synchronisation of project design and implementation	3	§5 Architecture, design-to-code traceability, status, gaps, demo

Contents

1. Problem Domain Identification and Analysis 3

1.1 Domain . . . 3

1.2 The problem . . . 3

1.3 Analysis of the problem . . . 3

1.4 Problem statement . . . 4

2. Review of Existing Systems and Project Feasibility 5

2.1 Existing systems . . . 5

2.2 Gap analysis and what Vani-Setu takes from each . . . 5

2.3 Feasibility study . . . 6

3. Objectives and Methodology of the Proposed Work 7

3.1 Objectives . . . 7

3.2 Methodology: the seven-stage pipeline . . . 7

3.3 Dataset . . . 8

3.4 Training and evaluation protocol . . . 9

3.5 Tools and technologies . . . 9

4. Relevance of Algorithms and Techniques 10

4.1 Techniques and why they fit . . . 10

4.2 How the SVM works and why it suits this problem . . . 10

4.3 Results: comparing five classifiers . . . 11

4.4 Error analysis . . . 12

4.5 Preliminary experiments for the proposed improvements . . . 13

5. Synchronisation of Project Design and Implementation 14

5.1 System architecture . . . 14

5.2 Design-to-code traceability . . . 15

5.3 API design . . . 16

5.4 Gaps between design and implementation, and the fix for each . . . 16

5.5 Demonstration plan for this evaluation . . . 16

References 17

MILESTONE 1

1. Problem Domain Identification and Analysis

1.1 Domain

Vani-Setu sits where **assistive technology**, **computer vision** and **machine learning** meet. The task is *sign language recognition*: turning hand shapes seen by a camera into text and speech, so that a deaf or speech-impaired person can talk with someone who does not know sign language. In ML terms this is a supervised multi-class classification problem on time-ordered visual data.

1.2 The problem

Sign language is a complete language with its own grammar, not a coded form of English or Hindi. India's 2011 Census recorded about **5.07 million** people with a hearing disability and about **2.0 million** with a speech disability [1]. Very few hearing people know Indian Sign Language (ISL), and trained interpreters are scarce: a 2021 list compiled for the Indian Sign Language Research and Training Centre (ISLRTC) named only **325** interpreters for the whole country [1].

As a result, a deaf person at a hospital, bank, railway counter, office or classroom usually depends on writing notes, on a family member, or on an interpreter being free. This is slow, loses detail, and takes away privacy and independence.

1.3 Analysis of the problem

Who is affected

Stakeholder	Need
Deaf and speech-impaired signers	Be understood by hearing people in everyday places, without an interpreter
Hearing people (staff, teachers, family)	Understand basic signs and fingerspelled words in real time
Learners of sign language	Practise letters and get instant feedback
Schools and public offices	A low-cost tool that runs on equipment they already have

Why the problem is technically hard

- **Variation between people:** hand size, skin tone, distance from the camera, and left- or right-handedness change what the camera sees.
- **Look-alike signs:** several letters differ only by one finger position (for example D/X, P/Z, R/U/V in ASL).
- **Motion:** letters such as J and Z, and most word signs, are defined by movement, not one pose.
- **Two hands and the face:** most ISL signs use both hands, and facial expression carries grammar.
- **Real time:** recognition must keep up with live video (around 15–30 frames per second) on an ordinary laptop.
- **Noisy frames:** single-frame predictions flicker, so the output must be stabilised before it is shown.
- **Limited data:** public ISL data is small compared with ASL data, and most of it is video rather than labelled hand poses.

Scope of this project

In scope	Out of scope (for now)
Fingerspelling A–Z (ASL) from one hand	Full sentence-level sign language grammar
A set of common one-handed word signs (ASL and simplified ISL)	Facial expression and body pose
Real-time recognition in a web browser with a laptop webcam	Special hardware (gloves, depth cameras)
Sentence building, speech output, translation into 25+ languages	Text-to-sign (avatar) direction
Planned: motion letters J/Z and two-handed ISL signs	Certified interpreting for legal or medical use

1.4 Problem statement

Millions of deaf and speech-impaired people in India communicate mainly through sign language, but very few hearing people understand it, and the country has only a few hundred trained interpreters. As a result, everyday conversations at hospitals, banks, offices and classrooms depend on writing notes or on an interpreter being available. Existing automatic translators either need special hardware such as sensor gloves or depth cameras, run on costly cloud services that upload a user's video, or support only American Sign Language and English.

Vani-Setu ("voice bridge") aims to build a low-cost, real-time translator that turns signs into text and speech, running in a web browser with an ordinary laptop webcam. MediaPipe Hand Landmarker will locate 21 key points on the hand in each frame, on the user's own device. These points will be adjusted for hand position, hand size and left or right hand, then turned into features that include joint angles and fingertip distances. Several machine learning models will be trained on these features and compared for fingerspelling A–Z and common phrases: a Support Vector Machine, classical baselines (Logistic Regression, KNN, Random Forest) and a neural network (multi-layer perceptron). A sequence model (LSTM) is planned for moving signs such as J and Z. Models will be tuned with cross-validation and evaluated using accuracy, macro F1-score, confusion matrices, and a separate test set recorded by new signers. Recognised words will form sentences that the app speaks aloud and translates into 25+ languages, including Hindi.

MILESTONE 2

2. Review of Existing Systems and Project Feasibility

We reviewed six existing systems and datasets that represent the main approaches: sensor gloves, multi-camera rigs, avatar apps, image-based deep learning, and landmark-based recognition. None of them gives a free, hardware-free, privacy-preserving sign-to-speech tool aimed at Indian users.

2.1 Existing systems

System	Approach	Strengths	Limitations for our users
SignAloud gloves, Univ. of Washington, 2016 [2]	Sensor gloves track hand position and movement; data sent over Bluetooth and spoken aloud	Won a Lemelson-MIT Student Prize; no camera needed	Needs special gloves; misses facial expression and body language; criticised by deaf users as a one-way bridge
SignAll (commercial) [3]	Originally multiple calibrated depth cameras and coloured-marker gloves; 2021 SDK moved to MediaPipe hand tracking	Large training set (300,000+ videos); developer SDK	ASL only; commercial; original setup was expensive and fixed to one place
Hand Talk app [4]	3D avatar "Hugo" signs out text or speech	Free mobile app; Libras and ASL	Opposite direction (text → sign); cannot read a signer
Image CNN interpreter (harshbg, HackUNT-19, 2019) [5]	Hand histogram segmentation + Keras CNN on cropped hand images	44 ASL characters; >95% reported accuracy	Sensitive to lighting and background; needs a calibrated histogram; heavy model
INCLUDE dataset, IIT Madras / AI4Bharat, 2020 [6]	ISL video dataset with deep-learning baselines	263 ISL word signs, 4,287 videos; first large ISL benchmark	A research dataset, not a usable app; video models are heavy for a laptop
PopSign ASL / Google ISLR, 2023 [7]	Smartphone videos converted to MediaPipe landmarks; landmark-sequence models	250 signs, 47 deaf signers, 210,000+ examples	ASL only; built for an educational game, not conversation

2.2 Gap analysis and what Vani-Setu takes from each

Gap in existing systems	How Vani-Setu addresses it
Special hardware (gloves, depth cameras)	Uses only an ordinary laptop webcam
Video sent to the cloud	Hand tracking runs inside the browser; only 63 numbers per frame reach the server
ASL and English only	Adds simplified ISL signs and translation into 25+ languages, with an offline Hindi fallback
Image CNNs break with lighting and background	Classifies hand landmarks, which are much less affected by background
Heavy video models	Light classical ML (1 MB SVM model); a neural network is added only where it helps
No way to add new signs	Users can record samples and retrain the model from the browser

From SignAll and PopSign we adopt the idea that **landmarks**, not raw pixels, are the right input for a light, real-time system. From the image-CNN project, which our code base started from, we keep the goal of

real-time fingerspelling but replace its segmentation-plus-CNN pipeline. From INCLUDE we take the ISL vocabulary direction for future work.

2.3 Feasibility study

Type	Assessment	Evidence
Technical	Feasible	MediaPipe Hand Landmarker gives 21 3D points per hand and runs in the browser [8]. Our SVM classifies one frame in about **0.55 ms** on a CPU (measured over 300 calls), so classification adds almost nothing to the per-frame budget. Model size: 1.05 MB (SVM) + 7.8 MB (hand model).
Economic	Feasible at zero cost	All tools are free and open source (Python, Flask, scikit-learn, MediaPipe). The dataset is MIT-licensed. Hosting uses Render's free tier. No hardware beyond a laptop.
Operational	Feasible	Users need only a browser and a webcam. The app works on localhost without internet, except for translation (Hindi has an offline fallback). A guest login lets examiners try it in seconds.
Schedule	Feasible with phasing	The core pipeline is already working (§5). Remaining work is split into must-do and stretch phases so the core is never at risk.
Legal and ethical	Feasible with care	Video stays on the device. Dataset licence is MIT and credited. The original open-source code base (MIT) is credited. The app is presented as a learning and communication aid, not a certified interpreter.

Risks and mitigation

Risk	Mitigation
Accuracy drops for new users (domain shift)	Normalise features; record a test set from new signers; add augmentation
Motion letters (J, Z) cannot be read from one frame	Sequence model over ~30 frames (planned stretch goal)
Free hosting sleeps when idle	Demo on localhost; use the hosted link only as a backup
Public retrain endpoint could be misused	Restrict data collection and retraining to local or admin use

3. Objectives and Methodology of the Proposed Work

3.1 Objectives

#	Objective	Measurable target
O1	Recognise fingerspelled letters A–Z in real time from one hand	≥ 95% held-out accuracy; ≥ 85% on new signers
O2	Make features robust to position, hand size and handedness	No accuracy loss when the hand moves nearer/farther or the left hand is used
O3	Compare classical ML and a neural network fairly	≥ 5 classical models + 1 MLP on identical features, with cross-validation
O4	Recognise common word signs with ML instead of fixed rules	10–15 phrases recorded from several people
O5	Keep the system real time on a normal laptop	≥ 15 frames per second; < 100 ms from sign to letter
O6	Turn signs into usable output	Sentence building, speech, translation into 25+ languages
O7	Report results honestly	Separate validation and test sets; per-class metrics; confusion analysis

3.2 Methodology: the seven-stage pipeline

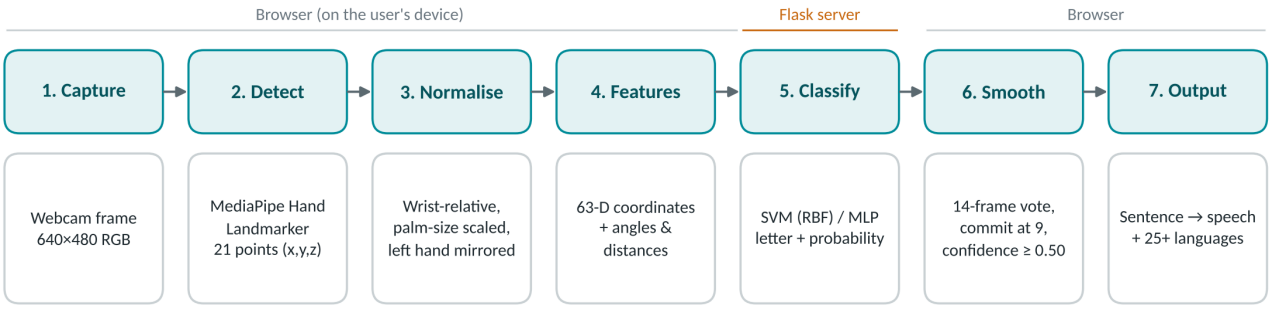


Figure 1. Recognition pipeline. Stages 1–4 and 6–7 run in the browser; stage 5 runs on the Flask server.

Stage 1–2: Capture and hand detection

The browser reads webcam frames with `getUserMedia`. MediaPipe Hand Landmarker (running mode VIDEO) finds one hand and returns 21 landmarks, each with normalised x, y and depth z. This reduces a 640×480×3 image (921,600 values) to 63 numbers, which is why the rest of the system can be light and fast.

Stage 3: Normalisation

Each point is expressed relative to the wrist (landmark 0), which removes where the hand is in the frame. We extend this by dividing by palm size and mirroring left hands:

$$p'_i = (p_i - p_0) \,/\, \lVert p_9 - p_0 \rVert \;,\; i = 0 \ldots 20$$

where p_0 is the wrist and p_9 is the middle-finger knuckle. For a left hand, x is multiplied by −1 so both hands look the same to the model.

Stage 4: Feature extraction

The base feature vector is the 21 × 3 = 63 normalised coordinates. We add 10 pairwise fingertip distances and 15 finger joint angles (angle at each joint between the two bones that meet there), giving 88 features. Angles do not change with hand size or rotation, which helps separate look-alike letters.

Stage 5: Classification

A `StandardScaler` + RBF Support Vector Machine pipeline predicts the letter and a probability. Five classical models and an MLP neural network are compared on the same features (§4). For moving signs, a sequence model over about 30 frames is planned.

Stage 6: Temporal smoothing

Single-frame predictions flicker. The app keeps the last 14 predictions and commits a letter only when at least 9 agree and the model's confidence is at least 0.50. After a commit, a 22-frame cool-down stops the same letter being added twice.

Stage 7: Output

Committed letters build words (with word-suggestion chips), words build a sentence, and the sentence can be spoken with the Web Speech API or translated into 25+ languages through the server.

3.3 Dataset

The letter model uses the **ASL Now Fingerspelling Landmarks** dataset (AxelS27, Hugging Face, MIT licence) [9]. Participants signed ASL letters to a camera and the MediaPipe Web Hand Landmarker stored 21 landmarks per sample, the same detector our app uses, so training and live data match. The dataset has **2,122 samples across 26 letters** (the Hugging Face page lists 44,562 rows because it counts each of the 21 points separately), stored in `dataset/landmarks.csv`.

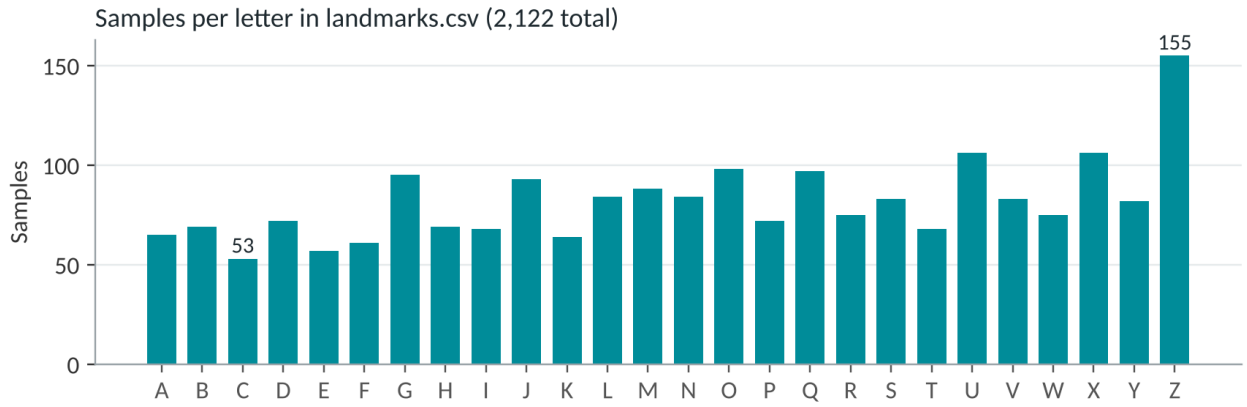


Figure 2. Samples per letter. The classes are uneven (C has 53, Z has 155), so we report macro F1 and use class weighting.

The dataset does not record who signed each sample, so a random split may place the same person in both training and test sets. For that reason we will also record our own test set from 4–5 classmates who are not in the training data.

3.4 Training and evaluation protocol

- **Split:** 70% training, 15% validation, 15% test, stratified by letter. The test set is used once, at the end.
- **Model selection:** 5-fold stratified cross-validation on the training set.
- **Hyperparameter tuning:** grid search over C ∈ {1, 3, 10, 30, 100} and γ ∈ {scale, 0.003, 0.01, 0.03} for the SVM.
- **Metrics:** accuracy, per-class precision, recall and F1, macro F1, confusion matrix.
- **Diagnostics:** learning curve (does more data help?) and validation curve (over- vs under-fitting).
- **Real-world check:** accuracy on the new-signer test set, reported separately from the dataset test set.
- **Speed:** MediaPipe time per frame, API round-trip time and frames per second.

3.5 Tools and technologies

Layer	Tools
Interface	HTML5, CSS3, JavaScript, Web Speech API
Computer vision	MediaPipe Tasks Vision (Hand Landmarker, 21 points)
Backend	Python 3, Flask, Flask-CORS, Gunicorn
Machine learning	scikit-learn (SVM, LR, KNN, RF, DT, MLP), NumPy, pandas, joblib, matplotlib; Keras/PyTorch for the sequence model
Data and accounts	CSV landmark files, SQLite via Flask-SQLAlchemy, Werkzeug password hashing
Language	deep-translator (Google Translate), offline Hindi dictionary
Deployment	Render (HTTPS), GitHub

4. Relevance of Algorithms and Techniques

Each technique below was chosen for a specific reason tied to our data: about 2,000 samples, 63–88 numeric features, 26 classes, and a hard real-time limit. Results are from `compare_models.py` and `train_classifier.py`, which use a stratified 75/25 split with seed 42.

4.1 Techniques and why they fit

Technique	Where used	Why it is relevant here	Course concept
MediaPipe Hand Landmarker (pre-trained CNN detector + landmark regressor)	Stage 2, browser	Turns a 921,600-value image into 63 numbers; trained on ~30,000 real hand images [8], so we do not need our own detector	Transfer learning, dimensionality reduction
Wrist-relative and palm-size normalisation, handedness mirroring	Stage 3	Removes position, scale and left/right differences that are not part of the sign	Feature engineering, invariance
Joint angles and fingertip distances	Stage 4	Capture finger bends directly; same value whatever the hand size	Domain-driven feature design
Standard scaling (z-score)	Before every distance-based model	SVM and KNN use distances, so all features must be on the same scale	Feature scaling
Support Vector Machine, RBF kernel	Stage 5 (main model)	Works well with small, dense, numeric data; the kernel draws curved boundaries between similar letters	Max-margin classifiers, kernel trick
Logistic Regression, KNN, Random Forest, Decision Tree	Baselines	Show that the chosen model wins by measurement, not by preference	Model comparison, bias–variance
Platt scaling (SVM probabilities)	Confidence bar, 0.50 cut-off	Lets the app reject unsure frames	Probability calibration
Multi-layer perceptron	Stage 5 (comparison)	A neural network on the same features; tests whether learned features beat hand-made ones	Backpropagation, ReLU, softmax, dropout
LSTM / 1D-CNN over frame sequences	Planned (J, Z, word signs)	Motion signs need time order, which a single frame cannot show	Sequence models
Sliding-window majority vote	Stage 6	Turns noisy per-frame guesses into stable letters	Ensemble over time, noise filtering
Stratified k-fold CV, grid search	Training	Reliable estimate on a small, imbalanced dataset; picks C and γ without touching the test set	Model selection, validation

4.2 How the SVM works and why it suits this problem

An SVM finds the boundary between two classes that leaves the widest possible margin to the nearest training points (the support vectors). With the RBF kernel, similarity between two hand poses x and x' is

$$K(x, x') = exp(-\gamma \| x - x' \|^2)$$

so poses that are close in feature space count as similar, and the boundary can curve around clusters. C controls how much the model is penalised for misclassifying training points (higher C fits the training data

more tightly); γ controls how far one sample's influence reaches. For 26 letters, scikit-learn combines one-vs-one SVMs. This suits our data because the dataset is small (where SVMs generalise well), the features are dense and numeric, and inference is fast enough for every video frame.

4.3 Results: comparing five classifiers

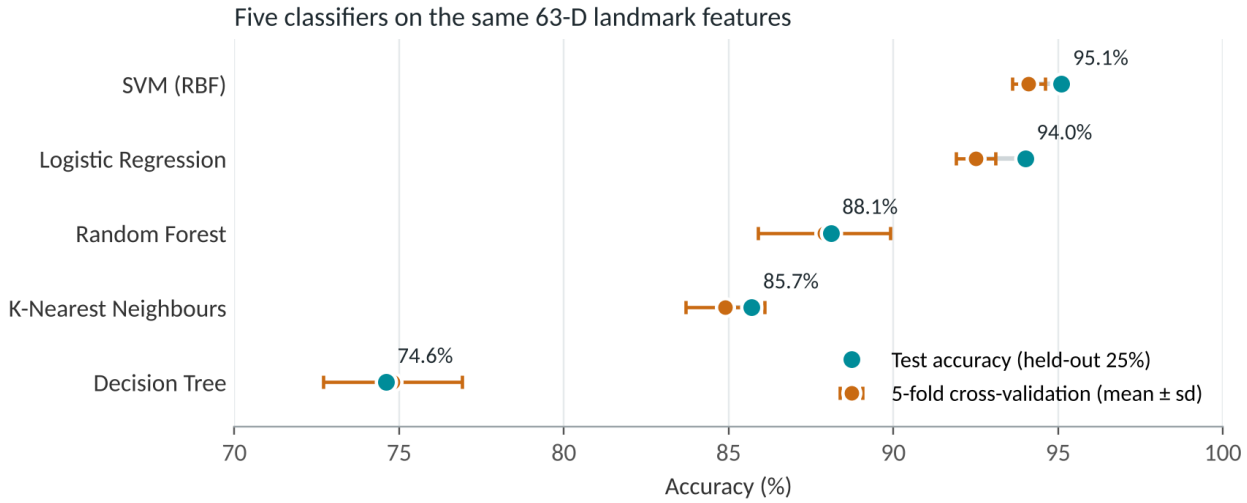


Figure 3. Test accuracy and 5-fold cross-validation on the same 63-D features (2,122 samples; 1,591 train / 531 test).

Model	Test accuracy	Macro F1	5-fold CV	Reading
SVM (RBF), C = 10	95.1%	0.953	94.1% ± 0.5%	Best and most stable; chosen
Logistic Regression	94.0%	0.940	92.5% ± 0.6%	Close: the classes are almost linearly separable
Random Forest	88.1%	0.879	87.9% ± 2.0%	Axis-aligned splits suit angles poorly
K-Nearest Neighbours	85.7%	0.853	84.9% ± 1.2%	Sensitive to distance in 63 dimensions
Decision Tree	74.6%	0.739	74.8% ± 2.1%	High variance: overfits a single tree

4.4 Error analysis

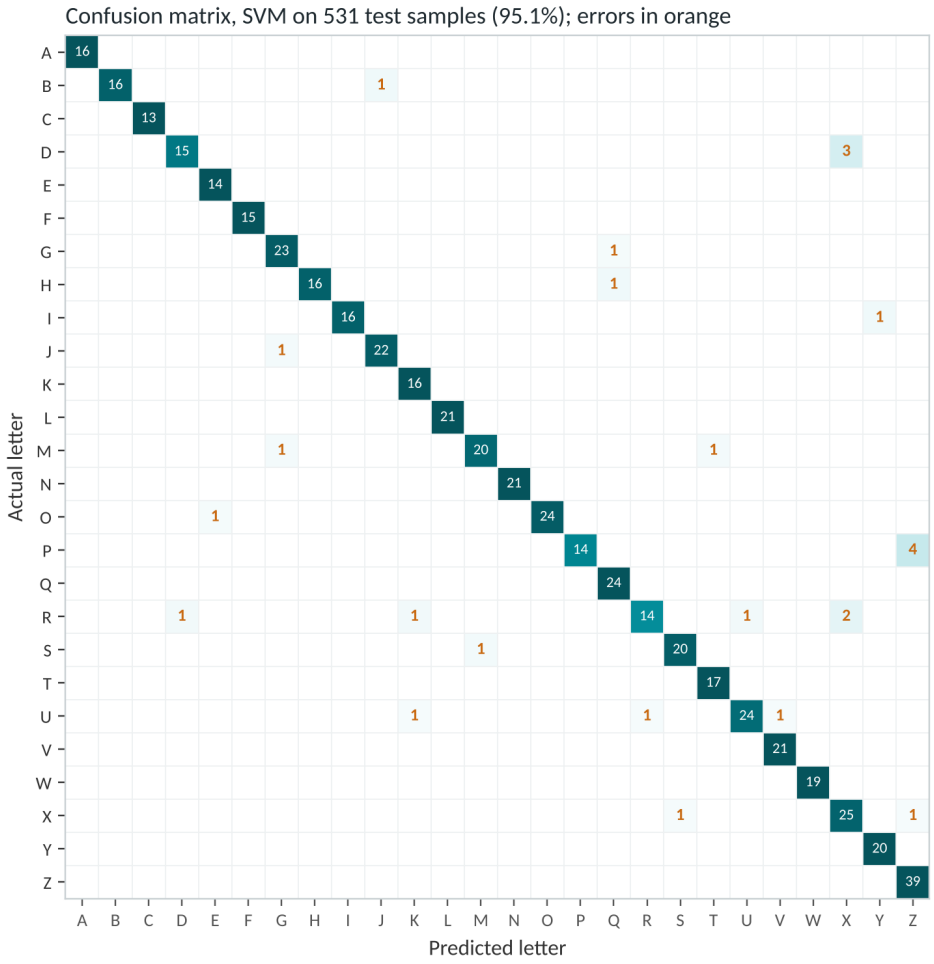


Figure 4. Confusion matrix of the SVM on the 531 test samples. 26 errors in total; orange numbers are mistakes.

The largest confusions are **P → Z** (4 samples), **D → X** (3) and **R → X** (2). D and X both use a single raised index finger (straight for D, hooked for X), and P and Z share an index-pointing shape, with Z defined by a movement that one frame cannot show. These are exactly the cases our planned features target: joint angles separate straight from hooked fingers, and a sequence model handles motion.

4.5 Preliminary experiments for the proposed improvements

To check that the proposed changes are worth building, we ran them offline on the same dataset and split. These experiments are not yet part of the live app; they will be integrated in the next milestone (§5.4).

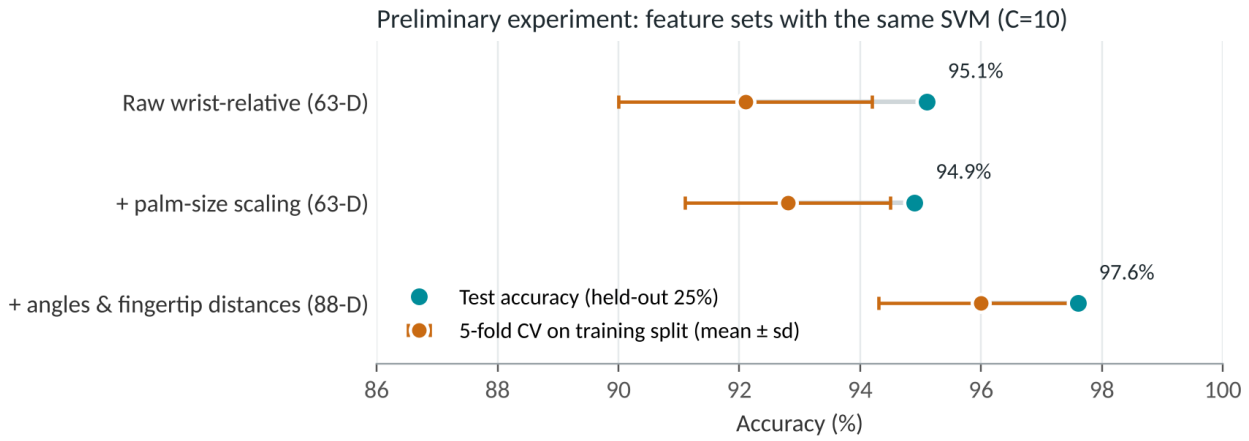


Figure 5. Same SVM, three feature sets. Adding joint angles and fingertip distances lifts test accuracy from 95.1% to 97.6% and CV from 92.1% to 96.0%.

Experiment	Result	What it tells us
Palm-size scaling (63-D)	94.9% test, 92.8% CV	No gain on this dataset (samples were recorded at similar distances); still needed for live use at varying distances
+ angles and fingertip distances (88-D)	97.6% test, 96.0% CV	Clear gain: half the errors removed (26 → 13)
Grid search for SVM on raw features	Best C = 100, γ = 0.01; CV 93.4% (vs 92.1%)	Tuning helps a little; feature design helps more
MLP 63 → 128 → 64 → 26, early stopping	93.0% test	Below the SVM on 2,122 samples; neural networks need more data
Label vs. confidence check	8 of 531 test samples	SVM `predict()` and `argmax(predict_proba())` disagree; fix by taking the label from the probabilities

5. Synchronisation of Project Design and Implementation

This section shows that each part of the design exists in code, where to find it, and its current status. Every "Done" item can be shown live during the evaluation.

5.1 System architecture

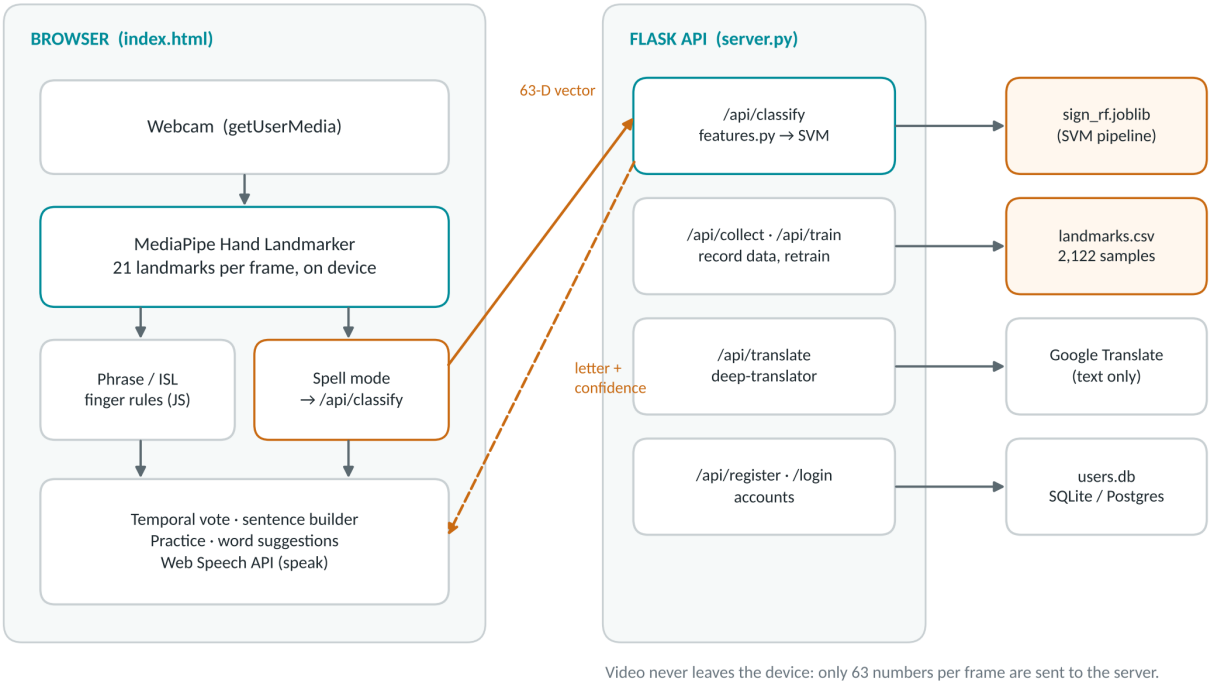


Figure 6. System architecture as implemented. Hand tracking stays in the browser; the Flask server classifies landmark vectors and serves translation, accounts and training.

5.2 Design-to-code traceability

Design module	Implemented in	Status	Evidence to show
Camera capture and hand tracking	`index.html` (MediaPipe Tasks Vision, VIDEO mode, 1 hand)	**Done**	Live landmarks drawn on the hand
Feature extraction (63-D, wrist-relative)	`features.py`	**Done**	Same function used for training and live classification
Palm-size scaling, handedness, angles	Offline experiment script	**Partly done**	Figure 5; to be moved into `features.py`
Classifier training and reports	`train_classifier.py`	**Done**	`reports/metrics.json`, confusion matrix
Model comparison (5 classifiers)	`compare_models.py`	**Done**	`reports/model_comparison.md`
Live classification API	`server.py` → `/api/classify`	**Done**	Confidence bar, ML MODEL badge
Temporal smoothing	`index.html` (buffer 14, commit at 9, cool-down 22)	**Done**	Stable letters while holding a sign
Phrase and ISL modes	`index.html` rules + `signs.py` (15 ASL, 12 ISL signs)	**Partly done**	Works by rules; ML version planned
Data recording and retraining	`/api/collect`, `/api/train`	**Done**	Retrain button hot-reloads the model
Accounts	`models_user.py` (SQLite), `/api/register`, `/api/login`	**Done**	Sign-up, sign-in, guest login
Translation and speech	`/api/translate`, Web Speech API, `signs.py` offline Hindi	**Done**	Speak and translate a sentence
Practice mode, word suggestions, sign guide	`index.html`	**Done**	Prompted letters with score
Deployment	`render.yaml`, `Procfile`, Gunicorn	**Done**	vani-setu.onrender.com
MLP neural network	Offline experiment	**Partly done**	93.0% test; integration planned
Sequence model for J, Z and word signs	—	**Planned**	—
New-signer test set	—	**Planned**	—

5.3 API design

Endpoint	Method	Input → output
`/api/classify`	POST	21 landmarks → letter + confidence
`/api/collect`	POST	Labelled landmarks → appended to the CSV dataset
`/api/train`	POST	Runs training, hot-reloads the model, returns accuracy
`/api/translate`	POST	English sentence + target language → translated text
`/api/register`, `/api/login`, `/api/guest-login`	POST	Credentials → session
`/api/health`, `/api/model-status`	GET	Readiness of the model and the hand landmarker

5.4 Gaps between design and implementation, and the fix for each

Gap found in review	Fix	Priority
Old CNN code from the original project is still in the repository, unused	Move to `legacy/`, credit the original project in the README	Must
Model file is named `sign_rf.joblib` but holds an SVM	Rename to `sign_svm.joblib` and update comments	Must
Features ignore hand size and handedness	Add palm-size scaling, mirroring and angle features (Figure 5)	Must
Model chosen by test-set accuracy	70/15/15 split; choose on validation and CV; test once	Must
Label and confidence can disagree (8 of 531 samples)	Take the label from `argmax(predict_proba)`	Must
No neural network in the app	Add the MLP to the comparison and the app	Must
Public site lets any guest collect data and retrain	Allow these only on localhost or for an admin account	Should
Phrase / ISL modes are rule-based, one hand only	Record phrase data and train; track two hands for ISL	Should
J and Z need motion	LSTM or 1D-CNN over ~30 frames	Stretch

5.5 Demonstration plan for this evaluation

- Run `./run.sh` and open `http://127.0.0.1:5001`; check `/api/health` shows ready.
- Sign in (or continue as guest) and start the camera; show landmarks on the hand.
- Spell mode: hold L, V, W and B; point out the confidence bar and the 14-frame vote.
- Spell the first letters of a word and complete it with a suggestion chip.
- Practice mode: sign 3–4 prompted letters and show the score.
- Phrase mode: Hello, Peace, OK; switch to ISL and show Namaste.
- Translate the sentence into Hindi and speak it.
- Open `reports/` to show the confusion matrix and model comparison; press Retrain to show training from the browser.

References

[1] S. Natarajan, "Widening human resources: Indian sign language updates," TaxGuru, Sept. 2021 (citing Census of India 2011 and ISLRTC). taxguru.in/corporate-law/widening-human-resources-indian-sign-language-updates.html

[2] "These gloves offer a modern twist on sign language" (SignAloud, N. Azodi and T. Pryor, University of Washington), WGBH News, 17 May 2016. wgbh.org/news/2016-05-17/these-gloves-offer-a-modern-twist-on-sign-language

[3] "SignAll SDK: Sign language interface using MediaPipe is now available for developers," Google Developers Blog, 14 Apr. 2021. developers.googleblog.com/2021/04/signall-sdk-sign-language-interface-using-mediapipe-now-available.html

[4] "Meet the Hand Talk sign language translator app," Hand Talk. handtalk.me/en/blog/meet-the-hand-talk-sign-language-translator-app

[5] H. Gupta, "Sign Language Interpreter using Deep Learning," HackUNT-19, 2019, MIT licence. github.com/harshbg/Sign-Language-Interpreter-using-Deep-Learning

[6] A. Sridhar, R. G. Ganesan, P. Kumar and M. Khapra, "INCLUDE: A Large Scale Dataset for Indian Sign Language Recognition," ACM Multimedia 2020. doi.org/10.1145/3394171.3413528

[7] T. Starner et al., "PopSign ASL v1.0: An Isolated American Sign Language Dataset Collected via Smartphones," NeurIPS 2023 Datasets and Benchmarks. Related: Google "Isolated Sign Language Recognition" competition, kaggle.com/competitions/asl-signs

[8] Google, "Hand landmarks detection guide," MediaPipe Solutions. developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker

[9] AxelS27, "ASL Now Fingerspelling Landmarks," Hugging Face dataset, MIT licence. huggingface.co/datasets/AxelS27/asl-now-fingerspelling

[10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research 12, pp. 2825–2830, 2011.